

要員通勤コスト比較ツール 利用マニュアル（使用説明書）

本書は、本ツールの利用者（営業・管理者）向けの操作手順をまとめたものです。

1. 本ツールでできること

客先現場までの **通勤時間・乗換回数・片道運賃（IC／きっぷ）・通勤定期代（1・3・6ヶ月）** を、待機中の全要員ぶん一括で比較します。利用の入口は3つあります。

入口	想定利用者	用途
LINE	営業	外出先からその場で現場の通勤コストを確認
コマンドライン	管理者	検証・CSV出力・名簿メンテ
API（IAM 認証）	連携	他システムからの呼び出し

2. LINE での使い方

- 公式アカウント「営業アシスタント」を友だち追加します。
- トークに **現場の駅名** をそのまま送信します（例：東京）。
- まず「収集中…ちょっと待ってね」と返り、数秒後に比較結果が届きます。

送信例と応答例

（あなた）東京

（Bot）収集中…ちょっと待ってね 🚦

（Bot）

- 現場「東京」への通勤コスト（待機中3名 / 定期代が安い順）
- ・ 拉拉（池袋→東京）：24分/乗換0回 IC253円 定期7,840円
経由：池袋→東京
 - ・ 山田太郎（横浜→東京）：27分/乗換0回 IC528円 定期15,600円
経由：横浜→東京
 - ・ 鈴木一郎（立川→東京）：64分/乗換1回 IC593円 定期19,580円
経由：立川→荻窪→東京

- ・ 駅名は **漢字・ひらがな・カタカナ・ローマ字** いずれも可（例：とうきょう / トウキョウ / tokyo）。

- 中国語表記（例：東京）も自動で日本語駅名へ変換して検索します。
- 現場確定済み（アサイン中）の要員は比較対象から自動的に除外されます。

3. コマンドライン（管理者向け）

```
# 単一区間
python commute.py 平井 東京

# 現場 → 待機中の全要員 一括比較（CSV 出力も可）
python query.py --site 東京 --strategy cheapest --csv out.csv

# 要員の追加・状態更新（住所からの最寄駅自動判定にも対応）
python staff_admin.py add E006 田中次郎 --address "東京都新宿区西新宿2-8-1" --dept 営業部
python staff_admin.py status E002 assigned --site 東京 # 現場確定（比較対象から外す）
```

戦略は `--strategy cheapest`（定期代が最安。同額なら時間最短）／ `--strategy fastest`（時間最短）。

4. 名簿（花名冊）の一括メンテナンス

CSV（UTF-8）を編集して流すだけで、要員の増・改・削を反映できます。

```
staff_id,name,nearest_station,address,department,status
E002,拉拉,池袋,,営業部,available
E011,高橋健,,東京都新宿区西新宿2-8-1,営業部,available
```

```
python staff_sync.py roster.csv --dry-run # 変更内容の確認
python staff_sync.py roster.csv --replace # 完全同期（ファイルに無い人は削除）
```

- `nearest_station` か `address` のどちらか必須。`address` は最寄駅に変換して保存し、住所自体は保存しません。
- `status` は `available`（待機）／ `assigned`（現場確定）。

5. よくある質問（FAQ）

質問	回答
初回の応答が遅い	初めての現場×駅は実データ取得のため数十秒かかることがあります。2回目以降はキャッシュで即時です。
返信が来ない	LINE 公式アカウント側の「応答メッセージ」を OFF、応答モードを Bot にしてください。

質問	回答
中国語で送ったら認識しない	簡体字は自動変換対象ですが、まれに変換できない駅は日本語駅名で送ってください。
結果の数値が前回と違う	取得時点のダイヤにより変動します。値は目安としてご利用ください。

6. 注意事項

- 本ツールは社内提案の比較を目的とした補助ツールです。最終的な金額・経路は公式の運賃案内でご確認ください。
- 暫定のデータ取得元を利用しています。正式運用では認可 API への切り替えを想定しています。